



In the world of big data, **Apache Spark** is a powerhouse, but understanding what's happening inside your jobs — especially in production — can be challenging. The built-in Spark UI is great for quick checks, but it lacks persistence, historical comparison, custom business metrics, and easy team-wide visibility.

So, I decided to build my own **metric logging system** for Spark applications and pair it with a custom dashboard UI. Here's the complete story of how I did it.

The Problem

- Spark jobs running on large clusters have no easy way to track long-term trends.

- Need for **application-level, stage-level, and custom business metrics** (e.g., records processed per domain, error rates by type).
- Teams wanted a single pane of glass instead of checking multiple Spark UIs or logs.
- Requirement for alerting and historical analysis.

Why Spark UI Was Not Enough

APACHE SPARK MONITORING CHALLENGES	
THE CHALLENGES	SPARK UI LIMITATIONS
1  Historical comparisons Performance over time is difficult to track.	Lost after retention period. (Data is transient.)
2  Multi-job visibility Lack of a single centralized view for all application stages.	No centralized view. (Requires context switching.)
 Business metrics Operational metrics are hard to translate to business value.	Not supported. (Focuses solely on low-level system metrics.)
 Proactive alerting Reactive only; difficult to set proactive thresholds.	Limited alerting capabilities. (Simple rule-based.)
5  Trend analysis Analysis of long-term trends is a manual process.	Difficult to perform. (Data is not indexed for complex queries.)
6  Cross-team sharing Sharing specific data slices is complex and limited.	Poor sharing capabilities. (Requires direct cluster access.)
7  Cost analytics Understanding cluster usage costs is completely absent.	Missing feature. (No built-in cost tracking.)
8  Custom KPIs Defining and monitoring custom KPIs is not possible.	Not available. (Offers predefined metrics only.)

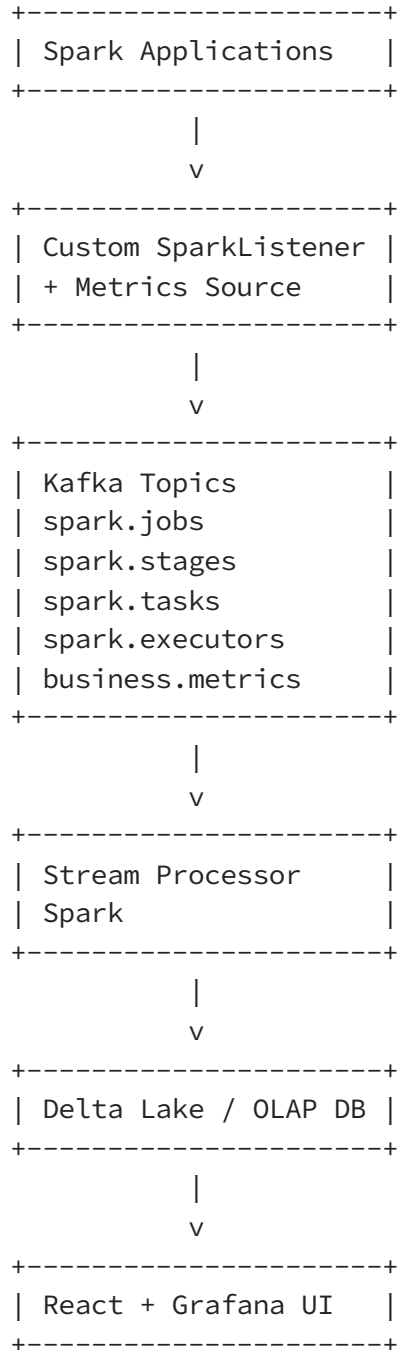
High-Level Architecture

I designed a system with these core components:

1. **Metric Collection Layer** (Inside Spark)
2. **Transport Layer** (Kafka or direct sink)

3. Storage Layer (Data lake / Delta Lake / ClickHouse / TimescaleDB)

4. UI / Dashboard Layer (Custom web app or Grafana)



Step 1: Collecting Metrics — Two Complementary Approaches

A. Extending Spark's Built-in Metrics System

Spark uses the **Dropwizard Metrics** library. You can create a custom Source and Sink.



I created a custom MetricsSource for business metrics:

```
class BusinessMetricsSource extends Source {  
  override val sourceName = "business"  
  val registry = new MetricRegistry()  
  
  val recordsProcessed = registry.meter("records.processed")  
  val processingLatency = registry.histogram("processing.latency.ms")  
  // ... more gauges, counters, timers  
}
```

Then registered it:

```
val metricsSystem = SparkEnv.get.metricsSystem  
metricsSystem.registerSource(new BusinessMetricsSource())
```

B. Custom SparkListener (The Real Powerhouse)

This was the most valuable part. I extended SparkListener to capture detailed events for jobs, stages, tasks, and executors.

```
class CustomMetricsListener extends SparkListener {  
  
  override def onJobStart(jobStart: SparkListenerJobStart): Unit = { ... }  
  
  override def onStageCompleted(stageCompleted: SparkListenerStageCompleted): Un  
    val metrics = stageCompleted.stageInfo.details // accumulator metrics, shuf  
    // Push to Kafka or database  
    sendToMetricsStore(stageCompleted, metrics)  
  }  
  
  override def onTaskEnd(taskEnd: SparkListenerTaskEnd): Unit = { ... }  
}
```

Added via config:

```
spark.extraListeners=com.mycompany.CustomMetricsListener
```

Or programmatically:

```
spark.sparkContext.addSparkListener(new CustomMetricsListener())
```

Step 2: Transport & Storage

I chose **Kafka** as the transport for low-latency, high-throughput streaming of metrics.

- A lightweight consumer (another Spark streaming job or Flink app) reads from Kafka.

- Transforms and writes to a partitioned table (e.g., Delta Lake):
- spark_app_metrics
- spark_stage_metrics
- spark_executor_metrics
- custom_business_metrics

I also experimented with **Prometheus + Grafana** using the PrometheusServlet sink for real-time scraping.

Step 3: Building the UI/Dashboard

I built a **React + Node.js** web application (with TypeScript) connected to the metrics database. Key features:

- **Application Overview Dashboard:** Success rate, duration trends, resource utilization over time.
- **Job/Stage Drill-down:** Detailed timelines, task distribution, skew detection.
- **Custom Business Metrics:** Domain-specific KPIs.
- **Comparison View:** Compare multiple runs of the same job.
- **Alerts:** Configurable thresholds (e.g., stage duration > 30 mins, spill > threshold).

Tech Stack for UI:

- Frontend: React, Recharts / Apache ECharts, TanStack Query
- Backend: FastAPI (Python) or Node/Express

- **Auth:** Internal SSO
- **Deployment:** Docker + Kubernetes

For faster iteration, I also set up **Grafana** dashboards connected to the same data source. Grafana handled most visualizations beautifully, while the custom UI was used for complex workflows and business-specific views.

Key Challenges & Solutions

- **Metric Volume:** High cardinality → Aggregated at ingestion where possible and used for downsampling for long-term storage.
- **Skew & Outliers:** Added specific detectors for data skew in stages.
- **Overhead:** Kept the listener lightweight; async sending of metrics.
- **Schema Evolution:** Used flexible schemas (JSON + structured columns) in Delta Lake.
- **Security:** Metrics contained no sensitive data.

Results

- Reduced debugging time by ~60–70%.
- Proactive identification of performance regressions.
- Better capacity planning and cost optimization.
- Teams across engineering now use the dashboard daily.

Lessons Learned

1. Start with SparkListener — it gives the richest data.
2. Combine system metrics + business metrics for real value.

3. Make the system pluggable (easy to add new metric types).
4. Persistence and historical view > real-time only.
5. Grafana is excellent for quick wins; custom UI for a polished product experience.

Data Engineering

Big Data

Big Data Analytics

Apache Spark

Data Engineer

**Written by Anurag Sharma**[Edit profile](#)

125 followers · 5 following

Data Engineering Specialist with 10+ exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!